

Executive Summary

OpenAI Gym (now largely superseded by Gymnasium) is a standard toolkit for developing and testing reinforcement learning (RL) agents. It provides **environments** (e.g. CartPole, FrozenLake, Atari games) that define *Markov Decision Processes (MDPs)* with states, actions, and rewards. Gymnasium is the actively maintained fork of Gym, fixing API issues (e.g. separating “terminated” vs “truncated” episode endings and a new `env.reset(seed=...)` interface) and adding modern features [8†L140-L148] [11†L167-L175]. To use Gym/Gymnasium effectively, one must understand core RL concepts (policies, value functions, on/off-policy learning, exploration/exploitation, returns and discounting, advantage, bootstrapping, function approximation, etc.) and key algorithms (tabular methods like Monte Carlo, Q-learning, SARSA; policy-gradient methods like REINFORCE, Actor-Critic, PPO, TRPO; deep Q-networks (DQN) and variants; model-based planning; offline RL basics). Practical skills include the Gym API (creating/resetting environments, understanding `action_space` / `observation_space`), using vectorized environments for faster data collection [26†L148-L156] [27†L151-L159], setting random seeds for reproducibility, monitoring/logging training (wrappers like `RecordVideo` or `Monitor`), customizing rewards or observations with wrappers [34†L15-L23], reward shaping, and evaluating agents. We outline a step-by-step learning path with hands-on projects (from simple tabular tasks to deep RL on continuous control), recommend libraries (Stable-Baselines3, RLlib, SpinningUp, etc.), primary references (official docs and seminal papers), and a timeline for learning over 1 week, 1 month, and 3 months.

1. OpenAI Gym vs. Gymnasium

OpenAI Gym is the original collection of benchmark environments and a standard interface (Python class `Env`) for single-agent RL [11†L167-L175]. It defines the agent-environment loop: the agent *observes* a state, *acts*, and receives a new state plus a *reward*, iterating until the episode ends. Gym provided many classic tasks (CartPole, MountainCar, Atari, etc.). **Gymnasium** is the community-maintained fork (by Farama Foundation) replacing Gym. It preserves the familiar API but fixes historical issues: e.g. Gym’s single boolean `done` flag has been split into `terminated` (episode success/failure) and `truncated` (time-limit cutoff) for clarity [7†L215-L220], and seeding now uses `env.reset(seed=...)` rather than `env.seed()` [12†L37-L45]. In practice, new projects should use Gymnasium for the latest features and compatibility, though almost all Gym tutorials apply with minimal changes (adapters exist for backwards compatibility [8†L140-L148]). Both provide environments with `action_space` and `observation_space` properties (Gym uses `gym.spaces` to define these) to specify valid actions and observations [9†L146-L154]. For example, CartPole is created by:

```
import gymnasium as gym
env = gym.make("CartPole-v1")           # Gymnasium API
obs, info = env.reset()                 # obs = [cart_pos, cart_vel,
pole_angle, pole_ang_vel]              # e.g. 0 (push cart left) or 1 (push
action = env.action_space.sample()
```

```
right)
next_obs, reward, terminated, truncated, info = env.step(action)
```

Here `env.reset()` initializes a new episode; `env.step(a)` returns `(obs, reward, terminated, truncated, info)` [11†L223-L231] [7†L215-L220]. Rendering can be done via `env.render()`, and *wrappers* (subclasses of `gym.Wrapper`) can modify observations, actions, or rewards in a modular way [33†L169-L178] [34†L15-L23].

2. The RL Problem Formulation (MDPs)

Reinforcement learning problems in Gym are modeled as *Markov Decision Processes* (MDPs). An MDP is defined by a set of **states** (the agent's observations of the environment), a set of **actions** available to the agent, a **transition model** (often unknown) that determines the probability of moving to a next state given a state and action, and a **reward function** assigning a numerical reward after each transition [16†L402-L409]. Formally:

- **States** (S) and observations: the environment's state (or observation) space. In Gym, the state might be fully or partially observable. For example, CartPole's state is a 4-dimensional vector of floats. Discrete tasks (e.g. FrozenLake) have a finite set of states, while continuous tasks (e.g. Pendulum) have continuous observation spaces. The Gym environment exposes `env.observation_space` (e.g. `Box`, `Discrete`, `Tuple` spaces) to describe this [9†L146-L154].
- **Actions** (A): choices the agent can make. Could be discrete (e.g. 2 actions in CartPole) or continuous (e.g. torque values in Pendulum). Gym's `env.action_space` describes valid actions (e.g. `Discrete(2)`, `Box(low, high)` etc.) [9†L146-L154].
- **Reward**: a scalar given at each step (or at episode end) indicating immediate performance. By design, the agent's goal is to **maximize cumulative reward** (the sum of rewards over time) [44†L717-L726] [48†L747-L756].
- **Episode**: a trajectory from an initial state until termination. Episodes can end naturally (terminal state) or by reaching a time limit. In Gymnasium, the loop breaks when either `terminated=True` or `truncated=True`. Each reset starts a fresh episode.
- **MDP specifics**: We usually assume the *Markov property*: the next state depends only on the current state and action. For example, the probability of the next state s' is $P(s'|s, a)$ and reward $R(s, a, s')$. Gym's `Env` class is designed to represent an (episodic) MDP [11†L167-L175].

Gym environments typically provide:

- **reset()**: initialize and return the initial observation.
- **step(action)**: apply an action, returning `(next_obs, reward, terminated, truncated, info)`.
- **render()**: (optional) visual display.
- **spaces**: `action_space` and `observation_space` define valid shapes/ranges [9†L146-L154].
- **wrappers**: e.g. `TransformObservation`, `RewardWrapper`, etc., allow injecting custom preprocessing or reward shaping [34†L15-L23]. For instance, a `RewardWrapper` can clip or reshape rewards. Gymnasium provides many built-in wrappers (see API docs) to normalize actions, stack frames, etc.

Overall, Gym/Gymnasium frames the RL loop: at each time step t , the agent sees state s_t (the observation), chooses action a_t , then the environment returns reward r_{t+1} and next state s_{t+1} . This repeats until the episode ends.

3. Core RL Concepts

To work effectively with Gym, you should master the following RL concepts:

- **Policy (π):** The agent's behavior strategy. A *policy* maps states to actions (deterministically or probabilistically). Formally, $\pi(a|s)$ is the probability of taking action a in state s [44†L707-L715]. The policy is what the agent learns. It can be represented as a table, a linear function, or a neural network. Example: in a simple game, a policy might say "if you see an obstacle on the left, steer right."
- **Value Function (V and Q):** Quantifies long-term quality of states or actions under a policy. The *state-value* $V^\pi(s)$ is the expected cumulative (discounted) reward from state s following policy π . The *action-value* $Q^\pi(s, a)$ is the expected return from state s taking action a and then following π . Intuitively, "value specifies what is good in the long run" [48†L747-L756]. The agent often learns value functions to evaluate and improve its policy. For example, Q-Learning learns $Q^*(s, a)$ to derive an optimal policy $\pi^*(s) = \arg \max_a Q^*(s, a)$.
- **Return and Discounting:** The *return* G_t is the cumulative future reward. Typically we use discounted return: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$, where $0 \leq \gamma \leq 1$ is the *discount factor* [52†L13-L21]. Discounting ensures the sum converges and emphasizes near-term rewards. For episodic tasks, one may use $\gamma = 1$, but for continuing tasks $\gamma < 1$ is common.
- **Advantage (A):** Measures the value of an action relative to the average at a state. $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$. Advantage functions reduce variance in policy gradient methods by centering rewards. (E.g. A2C/A3C use an advantage estimate.)
- **Model-based vs. Model-free:** In *model-based* RL, the agent builds or uses a model of the environment's dynamics (the transition $P(s'|s, a)$ or reward function) to plan (e.g. value iteration, trajectory rollouts). In *model-free* RL, the agent learns only from trial-and-error without explicit model: it directly estimates values or policies. Gym's standard algorithms (Q-learning, DQN, PPO, etc.) are model-free. *Example:* Using the known physics of a simulator is model-based, whereas learning from Monte Carlo rollouts in Gym is model-free [48†L779-L788].
- **On-policy vs. Off-policy:** An on-policy algorithm learns about the policy it is currently using to act. An off-policy algorithm learns about a (usually optimal) *target* policy while possibly following a different *behavior* policy to generate data. In short: *on-policy* methods evaluate or improve the same policy that is used to make decisions; *off-policy* methods can evaluate/improve a different policy from the one generating experience [49†L12-L18]. For example, SARSA is on-policy (it learns action-values for the current behavior policy), while Q-learning is off-policy (it learns values for the greedy policy while exploring with an ϵ -greedy behavior policy). This distinction affects stability and sample reuse.

- **Exploration vs. Exploitation:** The trade-off between *trying new actions* to discover potentially better rewards (exploration) and *using known good actions* to get higher immediate reward (exploitation). A common strategy is ϵ -greedy: with probability ϵ take a random action, else take the best-known action. Balancing exploration vs. exploitation is crucial (the “exploration–exploitation dilemma”) [16†L395-L403] . For example, a high ϵ in CartPole means more random actions to explore different balances; a low ϵ focuses on balancing based on current knowledge.
- **Bootstrapping:** In RL, *bootstrapping* refers to learning updates that use existing value estimates to update other estimates. Temporal-Difference (TD) methods like Q-learning or SARSA bootstrap because they update $Q(s,a)$ toward the immediate reward plus the discounted estimate of the next state’s value [53†L60-L64] . (In contrast, Monte Carlo methods wait until episode end to compute actual returns.) Bootstrapping often makes learning faster but can introduce instability when combined with function approximation.
- **Function Approximation:** Many environments have large or continuous state spaces, so we use function approximation (e.g. neural networks) instead of tabular lookup for value or policy. Deep RL (like DQN, PPO with neural nets) uses large function approximators. While powerful, combining function approximation with bootstrapping and off-policy updates can cause convergence issues (the so-called “*deadly triad*” [18†L203-L208]). In practice, stabilizing techniques (target networks, replay buffers, entropy regularization, etc.) are needed.
- **Stability/Convergence Issues:** Reinforcement learning can be unstable or diverge if not careful. The “deadly triad” of function approximation, bootstrapping, and off-policy learning can lead to divergence [18†L203-L208] . Other pitfalls include poorly scaled rewards, lack of feature normalization, or bad hyperparameters. (We discuss common debugging tips below.)

4. Key Algorithms (by Category)

Tabular Methods (Small MDPs): Assume finite states/actions; use tables for values.

- **Monte Carlo (MC) Control:** Learn value estimates by averaging returns after episodes. E.g., every time state s is visited in an episode, record the discounted return, and update $V(s)$ as an average of returns. Suitable for episodic tasks. *Complexity:* Depends on episode length; can be high sample cost. *Gym example:* Blackjack or FrozenLake, where state space is small enough to store V or Q .
- **Temporal-Difference (TD) Learning:** Learns on the fly by bootstrapping. Examples: **SARSA** (on-policy):

```
Initialize  $Q(s,a)$  arbitrarily.
For each episode:
    state = env.reset()
    choose action =  $\epsilon$ -greedy( $Q$ , state)
    while not done:
        next_state, reward, done = env.step(action)
        choose next_action =  $\epsilon$ -greedy( $Q$ , next_state)
```

```

    Q[state, action] +=  $\alpha$  [reward +  $\gamma$  Q[next_state, next_action] - Q[state,
    action]]
    state, action = next_state, next_action

```

- *On/off-policy*: On-policy (uses behavior policy ϵ -greedy).
- *Sample Efficiency*: Moderate.
- *Stability*: Generally stable for tabular, but needs exploring starts or adequate exploration.
- *Env examples*: Taxi-v3, FrozenLake.

- **Q-Learning** (Watkins' Q-learning, off-policy):

```

Initialize Q(s,a).
For each episode:
    state = env.reset()
    while not done:
        action =  $\epsilon$ -greedy(Q, state)
        next_state, reward, done = env.step(action)
        Q[state, action] +=  $\alpha$  [reward +  $\gamma$  max_a' Q[next_state, a'] - Q[state,
        action]]
        state = next_state

```

- *On/off-policy*: Off-policy (learns optimal policy independently of behavior policy).
- *Sample Efficiency*: Often higher than MC (uses each step for update).
- *Stability*: Can diverge with function approximation; tabular case converges under standard conditions.
- *Env examples*: Tabular games like FrozenLake-v1, Blackjack-v1 【21†L171-L179】 , CliffWalking.

Policy-Gradient Methods: Optimize parameterized policies by gradient ascent on expected return.

- **REINFORCE (Monte Carlo Policy Gradient)**: Learn a stochastic policy $\pi(a|s;\theta)$ using complete episode returns. Update rule for each time step t :

$$\theta \leftarrow \theta + \alpha \sum_t \gamma^t G_t \nabla_{\theta} \ln \pi(a_t | s_t; \theta).$$

- *On/off-policy*: On-policy (uses complete episodes with current policy).
- *Sample Efficiency*: Low (needs many episodes).
- *Stability*: High variance (improved by baselines).
- *Env examples*: Small tasks like CartPole, MountainCar (discretized or with softmax policy), Pendulum (with continuous policy).
- **Actor-Critic (A2C/A3C)**: Combine policy (actor) and value estimate (critic). The critic estimates $V(s)$ (usually with TD), the actor is updated by policy gradient with advantage. Asynchronous Advantage

Actor-Critic (A3C) uses multiple workers, while synchronous (A2C) runs in parallel. These methods are more sample-efficient than REINFORCE due to bootstrapping 【18†L142-L151】 .

- *Sample Efficiency*: Medium.
- *Stability*: Better than pure PG; on-policy by default (some variants incorporate off-policy buffers).
- *Env examples*: Continuous control (LunarLanderContinuous, BipedalWalker), Atari games (with softmax policies).
- **PPO (Proximal Policy Optimization) and TRPO (Trust Region Policy Optimization)**: Advanced on-policy policy-gradient algorithms that optimize a surrogate objective to constrain policy updates. PPO clips policy updates to avoid large changes, balancing stability and performance 【18†L153-L162】 . TRPO uses KL-divergence constraints.
- *Sample Efficiency*: Higher than vanilla PG but lower than off-policy.
- *Stability*: Very high (state-of-the-art).
- *Env examples*: Standard benchmarks (Atari games, MuJoCo tasks); popular in continuous control challenges.
- *Pseudocode snippet (PPO simplified)*: update policy by gradient of $\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)$ where r_t is the probability ratio and A_t is advantage.

Value-Based Deep RL: Use deep networks to approximate value/Q functions, usually with experience replay and target networks.

- **Deep Q-Network (DQN)**: Learns $Q(s,a;\theta)$ with a neural net. Uses replay buffer and separate target net for stability.
- *On/off-policy*: Off-policy.
- *Sample Efficiency*: High (reuses experience), but requires many samples.
- *Stability*: Challenging; stabilizations like double DQN, dueling networks are used.
- *Env examples*: Discrete action spaces only (CartPole, Atari games like Pong/Breakout, FrozenLake not too trivial for deep nets).
- *Variants*: DDQN (Double DQN), Dueling DQN, etc.
- **DDPG/SAC (Continuous Action)**: For continuous actions, extensions like DDPG (Deep Deterministic Policy Gradient) or SAC (Soft Actor-Critic) are used. These algorithms are off-policy actor-critic with value-function critics.
- *Sample Efficiency*: SAC is relatively high (off-policy), DDPG moderate.
- *Stability*: SAC is more stable (adds entropy regularization); DDPG can be unstable with noisy hyperparams.
- *Env examples*: Pendulum-v0, BipedalWalker, Roboschool and MuJoCo tasks.

Model-Based Methods: These build/predict environment dynamics. Examples range from simple Dyna-Q (learn a table model and do planning updates) to complex methods like MuZero. In practice, model-based RL (planning with learned models) is more advanced; Gym tasks with known physics (CartPole physics) are sometimes used with planning algorithms. Model-based RL can be very sample-efficient but is out of scope for beginners; focus first on model-free methods.

Offline (Batch) RL Basics: In offline RL, the agent learns from a fixed dataset of interactions without further environment queries. Algorithms like Batch Q-learning, Conservative Q-Learning (CQL), or BCQ adapt Q/Learning to static data. If interested, one can study papers (e.g. “Offline RL: Tutorial, Review, and Perspective” by Levine et al.). For beginners, focus on online methods first; offline RL is advanced.

Algorithm Comparison Table (sample efficiency and stability are qualitative, and action-space suitability is discrete vs continuous):

Algorithm	Type	Sample Efficiency	Stability	Policy Type	Action Space
Monte Carlo	On-policy (Tab.)	Low (needs full episodes)	Moderate (no bootstrapping)	On-policy	Discrete/ Any
SARSA	Tabular, TD	Moderate	High (on-policy)	On-policy	Discrete
Q-Learning	Tabular, TD	High (off-policy)	Moderate (off-policy)	Off-policy	Discrete
REINFORCE	Policy Gradient	Low	Low (high variance)	On-policy	Discrete/ Cont.
Actor-Critic (A2C/A3C)	Policy Gradient + Value	Medium	Medium/High (better than REINFORCE)	On-policy	Cont./Disc.
PPO	Policy Gradient	Medium	High (stable updates)	On-policy	Cont./Disc.
TRPO	Policy Gradient	Medium	High (trust-region)	On-policy	Cont./Disc.
DQN	Value-based (Deep)	High (experience replay)	Low/Medium (needs tricks)	Off-policy	Discrete
DDQN/ Dueling	Value-based	High	Higher (less bias)	Off-policy	Discrete
DDPG / SAC	Deep Actor-Critic	High	SAC: High, DDPG: Low	Off-policy	Continuous

(This table is illustrative; exact performance depends on implementation and tuning. “Stability” is relative: policy-gradient tends to be more stable, off-policy Q-learning more sample efficient but tricky [18†L179-L188] .)

5. Practical Skills & Libraries

Working with Gym/Gymnasium involves many practical considerations:

- **Environment API:** Use `gym.make()` to create environments; call `reset()` to start an episode and `step(action)` to advance. Understand the returned data structures (obs arrays, reward floats, done flags, info dict) [11†L223-L231]. Use `env.observation_space.sample()` or `env.action_space.sample()` to debug random inputs.
- **Vectorized Environments:** To collect experience faster, use Gymnasium's `SyncVectorEnv` or `AsyncVectorEnv` (via `gym.make_vec()`) to run many copies of the environment in parallel [26†L148-L156] [27†L151-L159]. This gives linear speed-up in samples/sec. For example:

```
envs = gym.make_vec("CartPole-v1", num_envs=8, vectorization_mode="sync")
obs_batch, info = envs.reset() # obs_batch shape: (8, obs_dim)
actions = [envs.single_action_space.sample() for _ in range(8)]
next_obs_batch, reward_batch, terminated, truncated, info =
envs.step(actions)
```

Note: training with vector envs may require adjustments (e.g. handling parallel episodes and combined done flags) [27†L151-L159].

- **Seeding and Reproducibility:** Control randomness via `env.reset(seed=...)` (Gymnasium) rather than the deprecated `env.seed()` [12†L37-L45]. Always set seeds for NumPy/PyTorch and the environment if you need reproducible results. Gymnasium's new interface allows per-episode seeds (helpful for deterministic debugging) [12†L43-L51].
- **Wrappers:** Gym provides many wrappers to modify environments without changing source code. Common wrappers include `TimeLimit` (to enforce episode length), observation/action transforms, `RecordVideo` to save gameplay videos [29†L268-L276], and monitoring wrappers (Stable Baselines3's `Monitor` logs episode rewards, lengths, etc.). For example, `RecordVideo(env, video_folder="videos/")` will record videos periodically [29†L268-L276]. Reward shaping (e.g. clipping or scaling rewards) can be done via a `RewardWrapper` [34†L15-L23]. Observation wrappers can flatten or stack frames for image inputs.
- **Logging & Evaluation:** Use TensorBoard, Weights & Biases, or Matplotlib to track training curves (episode reward vs episodes). Many libraries (e.g. Stable-Baselines3) have built-in logging callbacks. Evaluate policies by running *evaluation episodes* without exploration noise and averaging returns.
- **Curriculum/Domain Randomization:** For complex tasks, gradually increase difficulty (e.g. start with easier versions of the environment) or randomize environment parameters to improve generalization. Gym itself doesn't have built-in curriculum tools, but you can script progressive difficulty (e.g. changing gravity in LunarLander).

- **Hyperparameter Tuning:** RL has many hyperparameters (learning rate, γ , ϵ schedule, network architecture). Tools like Ray Tune or Optuna can automate search. Start with known defaults (PPO: $\gamma=0.99$, $\text{clip}=0.2$, etc. from original papers).
- **Monitoring & Debugging:** Watch rendered episodes for sanity. Common checks: reward scales (normalize if too large/small), gradient magnitudes (clip gradients if exploding), network outputs (action distributions should not saturate). If training is unstable, reduce learning rates or simplify the model.
- **Libraries:**
 - **Stable Baselines3 (SB3)** [38†L67-L75] : A widely-used PyTorch library with many implemented algorithms (DQN, PPO, SAC, etc.), built on Gym/Gymnasium. SB3 follows the Gym interface and provides utility wrappers (Atari preprocessing, action noise, replay buffers). Use it for reference implementations or quick baselines.
 - **RLlib (Ray):** Scalable RL library for training with distributed execution (TensorFlow/PyTorch support). Good for large experiments.
 - **Spinning Up (OpenAI):** Educational resource with readable implementations of core algorithms and tutorials [18†L142-L151] . Great for learning.
 - **Others:** TensorFlow Agents, CleanRL, Dopamine (for Atari), etc.
- **Environment Wrappers & Utilities:** Gymnasium's `gym.spaces` module (Box, Discrete, Tuple, Dict) defines how to sample valid actions/observations [9†L146-L154] . Use wrappers like `gym.wrappers.RecordVideo`, `TransformObservation`, `RewardWrapper` [34†L15-L23] to preprocess the environment. For example, clipping rewards to $[-1, 1]$ can stabilize some algorithms.
- **Evaluation Metrics:** Track mean episode return, episode length, success rate (for tasks with clear success criterion). Plot learning curves. Compare against known benchmarks (e.g. “solved CartPole” ~195 average reward).
- **Reproducibility Tips:** Fix random seeds (environment, NumPy, torch) [12†L37-L45] , use deterministic algorithms where possible (some GPU ops are nondeterministic), and log seeds alongside results. Use `gymnasium.Env.seed` (deprecated) or `env.reset(seed=123)` consistently.

6. Learning Path with Exercises

Week 1 – Foundations:

- **Goal:** Understand the RL loop and basic algorithms.
- **Concepts:** MDPs, policy/value, exploration vs exploitation, return/discount. Read “RL basics” (e.g. [16] or Spinning Up Ch.1).
- **Environments:** Start with very simple Gym tasks: CartPole-v1 (balance a pole), MountainCar-v0 (drive car up hill). Also try a tabular task like FrozenLake-v1 or Blackjack-v1 (toy-text environments).

- **Hands-on:** Write a simple *random agent* (sample actions) and log its performance. Try ϵ -greedy. Implement *Tabular Q-Learning* on a discrete environment (FrozenLake or Blackjack) from scratch using Python. Verify it learns a policy. Use `env.observation_space` and `env.action_space` to size your Q-table.

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", is_slippery=False)
Q = np.zeros((env.observation_space.n, env.action_space.n))
```

See Gymnasium tutorial for FrozenLake Q-learning [\[21†L175-L184\]](#) .

- **Deliverable:** Jupyter notebook implementing Q-Learning or SARSA on a simple env. Plots of episode reward over time.

Month 1 – Classic RL Algorithms:

- **Goal:** Implement and compare tabular TD methods; learn basic policy gradients; try simple neural nets.
- **Concepts:** Dive into SARSA (on-policy TD) and Q-Learning (off-policy TD) (Sutton & Barto Ch. 6). Understand Monte Carlo updates. Learn policy gradients basics (REINFORCE).
- **Environments:** FrozenLake (tabular), Blackjack (tabular), CartPole (with linear function approximator or shallow net).
- **Hands-on:** Use Gymnasium's tutorials (e.g. *Solving Frozenlake with Tabular Q-Learning* [\[21†L175-L184\]](#)) as guides. Implement REINFORCE (Monte Carlo policy gradient) on CartPole using a small neural network (output logits for 2 actions). Compare random vs trained agent.
- **Tools:** Try Stable-Baselines3's implementations (e.g. DQN or PPO) on CartPole/MountainCar to see baseline performance.
- **Deliverable:** Notebook showing tabular Q-Learning on one env, and a REINFORCE-trained agent on CartPole (with learning curve plots).

Month 2 – Deep RL and Continuous Control:

- **Goal:** Move to value-based deep RL and continuous-action problems.
- **Concepts:** Understand Deep Q-Network (DQN) – experience replay, target networks. Learn actor-critic (A2C) and policy optimization (PPO).
- **Environments:** CartPole and LunarLanderContinuous-v2 (Pendulum-v1) for continuous. Atari like Breakout or Pong (use Gym Atari with wrappers if ambitious).
- **Hands-on:** Use SB3 to train a DQN on CartPole and a PPO on LunarLander. Then try coding a simple DQN yourself (using PyTorch/Keras) on CartPole. Experiment with hyperparameters (network size, learning rate). Use vectorized envs to speed up. Monitor rewards.
- **Tools:** Stable-Baselines3 (try `PPO("MlpPolicy", env)`), Spinning Up examples.
- **Deliverable:** Report/note comparing DQN vs PPO vs A2C on two environments (e.g., CartPole vs Pendulum). Visualize policy (e.g. watch saved videos of agent). Possibly publish a short blog or GitHub readme summarizing findings.

Month 3 – Advanced Topics and Projects:

- **Goal:** Tackle complex tasks, explore cutting-edge RL.
- **Concepts:** Offline RL intro (learn from datasets), multi-agent? (optional), safety/Robustness, domain randomization/curriculum. Read advanced topics/papers (MuZero, off-policy PG). Study “Stable Baselines RL Tips” [38†L99-L104] for best practices.
- **Environments:** MuJoCo or PyBullet tasks (e.g. HalfCheetah, Ant) if compute allows. Complex Atari games. Robotics simulators if available (e.g. OpenAI Gym robotics).
- **Hands-on:** Choose a capstone project: e.g., “Train an agent to solve LunarLanderContinuous with SAC or PPO.” Or use an environment from RL Benchmarks (like Procgen, DM Control via gym wrappers). Implement an offline RL algorithm using a public dataset (e.g. D4RL) if interested. Contribute a Gym wrapper or environment.
- **Deliverable:** A mini-project writeup or code repository. E.g.: “PPO on BipedalWalker – tuning report” or “Offline DQN on TrafficLight domain”. Ensure to include experiments, training curves, and analysis.

7. Resources & Tools

- **Official Documentation:**
 - *Gymnasium docs* – comprehensive API guide and tutorials [11†L167-L175] [33†L143-L152] .
 - *OpenAI Gym Quickstart* (older, similar to Gymnasium).
 - *Stable Baselines3 Docs* – for code examples of algorithms [38†L67-L75] .
- **Textbooks & Papers:**
 - *Sutton & Barto, “Reinforcement Learning: An Introduction”* (2nd ed., 2018) – foundational theory and algorithms (free PDF available) [44†L707-L716] [48†L747-L756] .
 - *Spinning Up in Deep RL (OpenAI)* – tutorial series with explanations and code [18†L142-L151] [18†L179-L188] .
 - *David Silver’s RL Course* (videos/slides) and *DeepMind papers* (DQN2015, TRPO2015, PPO2017, etc.).
 - *RL Bootcamp (Haarnoja & Schaul 2018)* – advanced topics.
- **Libraries:**
 - *Stable Baselines3* [38†L67-L75] (PyTorch-based implementations).
 - *Ray RLlib* (scalable RL).
 - *Gym Retro* for classic games, *Gym Atari* (for Atari with wrappers).
- **Tutorials & Examples:**
 - Gymnasium training tutorials (FrozenLake Q-learning, Blackjack Q-learning) [21†L171-L179] .
 - *Atari walkthroughs* (e.g. Ale’s Arcade library).
 - Community notebooks on Colab (TensorFlow Agents tutorials, etc).
- **Community & Benchmarks:**
 - *Reinforcement Learning Repositories:* `ziggy-orange/RL-Baselines3-Zoo`, *OpenAI Baselines*, *HuggingFace RL Zoo*.
 - *Online courses:* Coursera’s “Deep Reinforcement Learning Specialization” by UWash.
 - *Discussions:* StackOverflow RL tag, Gym GitHub issues, `r/reinforcementlearning`.
 - **Hyperparameter Tuning:** Ray Tune, HyperOpt, Optuna.
 - **Useful Tools:** GPU/TPU for deep RL; TensorBoard or `csv_logger`; Python debugging (print statements, logging); environment replay recorders (Gym’s `RecordVideo` [29†L268-L276]).

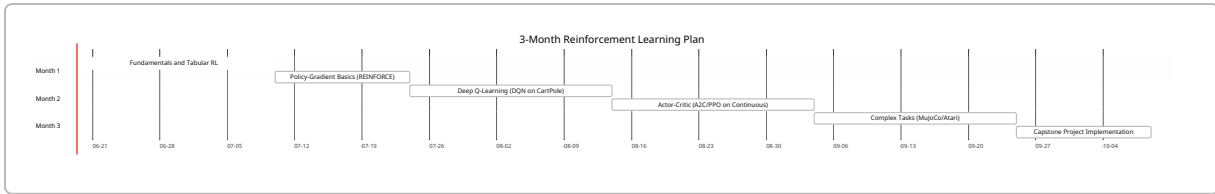
8. Pitfalls & Troubleshooting Checklist

- **State/Action Mismatch:** Ensure your state/action representations match `env.observation_space` and `env.action_space`. Mismatched shapes cause subtle bugs.
- **Not Resetting Env Properly:** After an episode ends, always call `env.reset()` again before `step()`. New Gym syntax returns `(obs, info)`.
- **Ignoring Truncation:** Handle both `terminated` and `truncated` flags. `done = terminated or truncated`. Otherwise episodes may not end correctly.
- **Unseeded RNG:** Random results! Always set random seeds for reproducibility [12†L37-L45]. Remember to seed the environment on reset.
- **Reward Scaling:** If rewards are too large/small, normalize or clip them. Many tutorials use reward clipping (especially in Atari).
- **Exploration Failure:** Using $\epsilon=0$ (greedy) from the start can cause the agent to get stuck in a suboptimal behavior. Start with a high ϵ and decay.
- **Algorithm Specifics:**
 - *Q-Learning/SARSA:* Verify that Q-table updates correctly (shape, indices). Off-by-one in indexing discrete states is common.
 - *Function Approximation:* If using neural nets, check for divergence: clip gradients, reduce learning rate, add entropy regularization (for PG methods).
- **Non-Stationary Data (Replay):** In DQN, ensure you shuffle or use a large replay buffer. Training on consecutive correlated frames without buffer leads to divergence.
- **Silent Failures:** Check for NaNs or infinities in losses. SB3 has `gradient_clip_val` options and NaN guards.
- **Stability Checks:** If learning collapses, try simpler tasks (CartPole) to debug your code, then move to harder tasks.
- **Convergence Issues:** As noted in literature, combining function approximation, bootstrapping, and off-policy can diverge [18†L203-L208]. If you see oscillating performance, consider on-policy methods or double learning tricks.
- **Environment Bugs:** Sometimes the environment itself changes (Gym updates). Lock versions or read release notes to avoid API mismatches [7†L215-L220].

9. Timeline & Learning Plan

- **1-Week Plan:** Learn RL fundamentals and Gym basics. *Milestones:* Code and run a random policy in CartPole; implement Q-Learning on FrozenLake; understand policy/value definitions. *Deliverables:* A notebook with simple experiments (random vs Q-learning) and annotated results (returns vs episodes).
- **1-Month Plan:** Master classic algorithms and small neural nets. *Milestones:* Successfully train agents with tabular (Q-Learning, SARSA) and simple PG (REINFORCE) on CartPole/MountainCar. Familiarize with SB3 by training a PPO or DQN on these tasks and analyzing hyperparameters. *Deliverables:* Short report (or README) comparing Q-Learning vs SARSA vs REINFORCE on chosen environments, with plots.
- **3-Month Plan:** Tackle deep RL and advanced topics. *Milestones:* Train deep networks on continuous control (e.g. LunarLanderContinuous, BipedalWalker) using PPO/SAC. Experiment with vectorized envs and logging. Begin reading advanced papers (PPO, SAC, MuZero). Possibly try offline RL with a

dataset. Develop a capstone project (e.g. an RL agent for a chosen environment, or contribute code).
Deliverables: A cohesive project (GitHub repo or presentation) showcasing a learned agent on a challenging task, along with a discussion of algorithm choice, results, and lessons learned.



(The Gantt chart above illustrates overlapping phases. Start dates are illustrative, beginning at 2026-06-19.)